

COMP 3250

DATA ANALYTICS

Linear Models for
Multiclass
Classification

AGENDA



Linear models for multiclass classification



Strengths, weaknesses, and parameters



Naive Bayes Classifiers

LINEAR MODELS FOR MULTICLASS CLASSIFICATION



Many linear classification models are for binary classification only and don't extend naturally to the multiclass case (except for logistic regression).



A common technique to extend a binary classification algorithm to a multiclass classification algorithm is the *one-vs.-rest approach*.



In the *one-vs.-rest approach*, a binary model is learned for each class that tries to separate that class from all of the other classes, resulting in as many binary models as there are classes.

LINEAR MODELS FOR MULTICLASS CLASSIFICATION

Having one binary classifier per class results in having one vector of coefficients (w) and one intercept (b) for each class.

The class for which the result of the classification confidence formula given here is highest is the assigned class label:

$$w[0] * x[0] + w[1] * x[1] + \dots + w[p] * x[p] + b$$

APPLYING ONE-VS.-REST METHOD TO A SIMPLE THREE-CLASS CLASSIFICATION DATASET

- `from sklearn.datasets import make_blobs`
- `X, y = make_blobs(random_state=42)`
- `mglearn.discrete_scatter(X[:, 0], X[:, 1], y)`
- `plt.xlabel("Feature 0")`
- `plt.ylabel("Feature 1")`
- `plt.legend(["Class 0", "Class 1", "Class 2"])`

LET'S TRAIN LINEAR SUPPORT VECTOR CLASSIFIER ON THE DATASET

```
linear_svm =  
LinearSVC().fit(X, y)
```

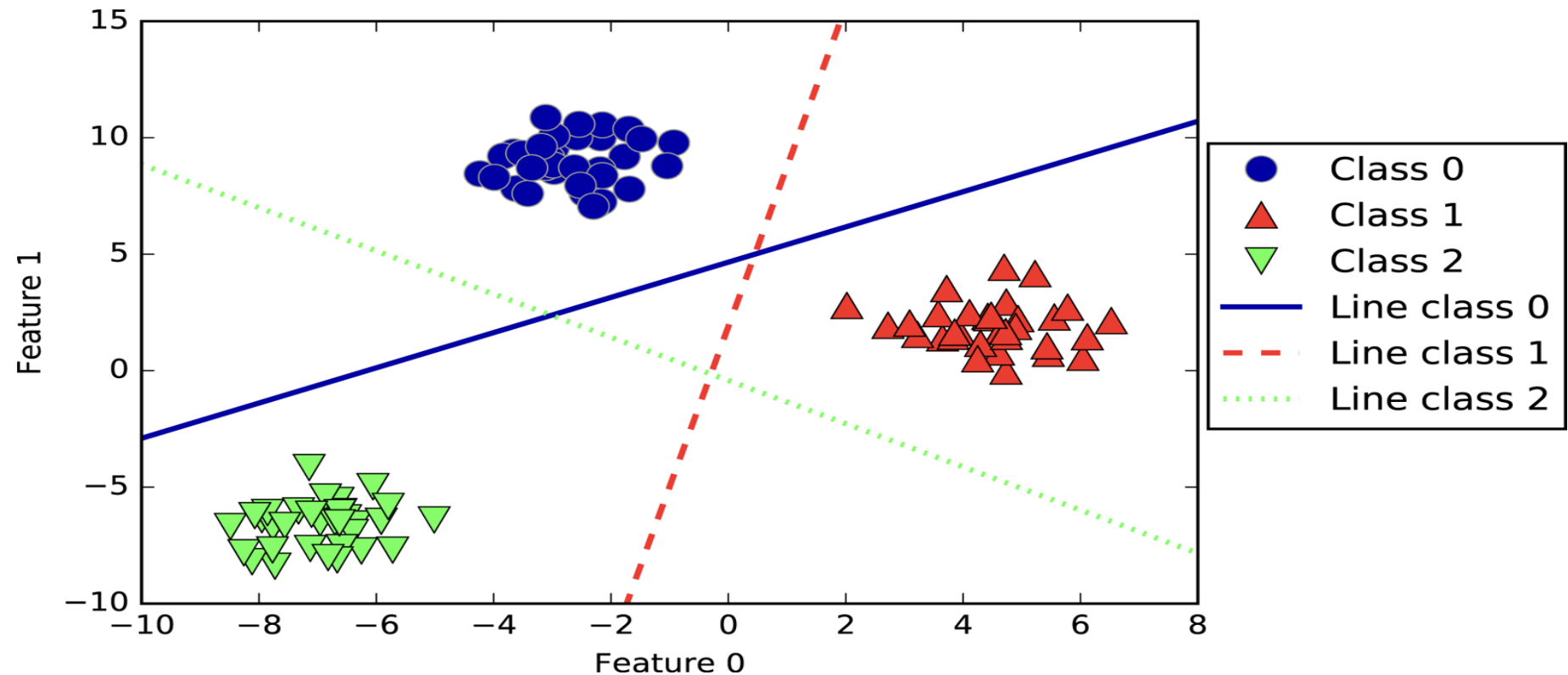
```
print("Coefficient shape: ",  
linear_svm.coef_.shape)
```

```
print("Intercept shape: ",  
linear_svm.intercept_.shape)
```

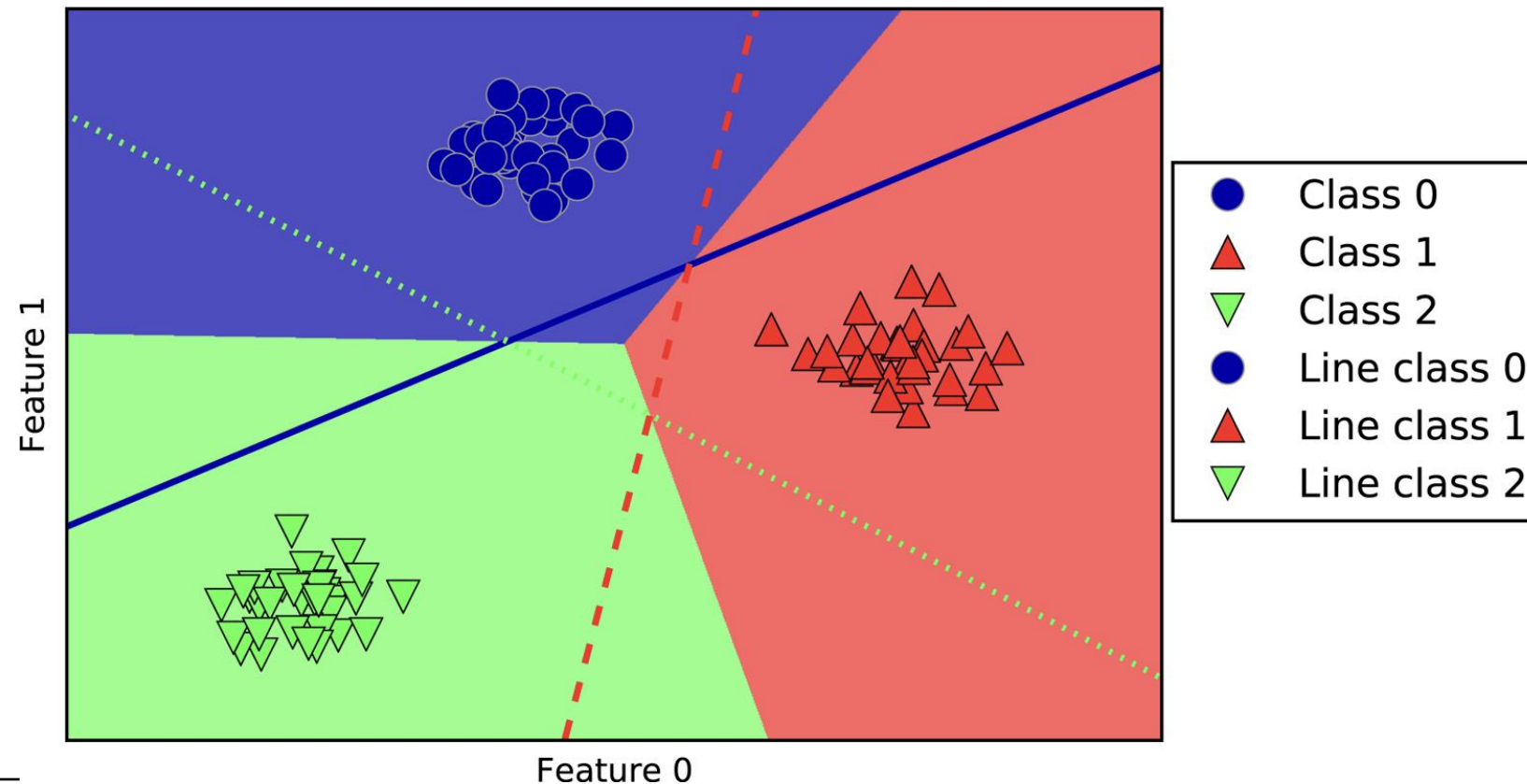
LET'S VISUALIZE THE LINES GIVEN BY THE THREE BINARY CLASSIFIERS

- `mglearn.discrete_scatter(X[:, 0], X[:, 1], y)`
- `line = np.linspace(-15, 15)`
- `for coef, intercept, color in zip(linear_svm.coef_, linear_svm.intercept_,`
- `['b', 'r', 'g']):`
- `plt.plot(line, -(line * coef[0] + intercept) / coef[1], c=color)`
- `plt.ylim(-10, 15)`
- `plt.xlim(-10, 8)`
- `plt.xlabel("Feature 0")`
- `plt.ylabel("Feature 1")`
- `plt.legend(['Class 0', 'Class 1', 'Class 2', 'Line class 0', 'Line class 1',`
- `'Line class 2'], loc=(1.01, 0.3))`

LET'S VISUALIZE THE LINES GIVEN BY THE THREE BINARY CLASSIFIERS



MULTICLASS DECISION BOUNDARIES DERIVED FROM THE THREE ONE-VS.-REST CLASSIFIERS



MULTICLASS DECISION BOUNDARIES DERIVED FROM THE THREE ONE-VS.-REST CLASSIFIERS

- `mglearn.plots.plot_2d_classification(linear_svm, X, fill=True, alpha=.7)`
- `mglearn.discrete_scatter(X[:, 0], X[:, 1], y)`
- `line = np.linspace(-15, 15)`
- `for coef, intercept, color in zip(linear_svm.coef_, linear_svm.intercept_,`
- `['b', 'r', 'g']):`

```
plt.plot(line, -(line * coef[0] + intercept) / coef[1], c=color)
plt.legend(['Class 0', 'Class 1', 'Class 2', 'Line class 0', 'Line class 1',
'Line class 2'], loc=(1.01, 0.3))
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
```



STRENGTHS, WEAKNESSES, AND PARAMETERS

STRENGTHS OF LINEAR MODELS



- The main parameter of linear models is the regularization parameter, called alpha in the regression models and C in LinearSVC and LogisticRegression.
- Large values for alpha or small values for C mean simple models.
- The other decision you have to make is whether you want to use L1 regularization or L2 regularization
- If you assume that only a few of your features are actually important, you should use L1. Otherwise, you should default to L2.
- L1 can also be useful if the interpretability of the model is important. As L1 will use only a few features, it is easier to explain which features are important to the model, and what the effects of these features are.

STRENGTH OF LINEAR MODELS

- Linear models are very fast to train, and also fast to predict, scale to very large datasets and work well with sparse data.
- If your data consists of hundreds of thousands or millions of samples, you might want to investigate using the `solver='sag'` option in `LogisticRegression` and `Ridge`, which can be faster than the default on large datasets.
- Linear models make it relatively easy to understand how a prediction is made using the formulas we saw earlier for regression and classification.



WEAKNESSES



it is often not entirely clear why coefficients are the way they are. This is particularly true if your dataset has highly correlated features; in these cases, the coefficients might be hard to interpret.



Linear models often perform well when the number of features is large compared to the number of samples



They are also often used on very large datasets simply because it's not feasible to train other models. However, in lower-dimensional spaces, other models might yield better generalization performance.

METHOD CHAINING

- *# instantiate model and fit it in one line*
- *logreg = LogisticRegression().fit(X_train, y_train)*
- This concatenation of method calls (here `__init__` and then *fit*) is known as **method chaining**.
- Another common application of method chaining in *scikit-learn* is to *fit* and *predict* in one line:
- *logreg = LogisticRegression()*
- *y_pred = logreg.fit(X_train, y_train).predict(X_test)*

Model instantiation, fitting, and predicting in one line:

- *y_pred = LogisticRegression().fit(X_train, y_train).predict(X_test)*
- **This very short variant is not ideal, though. A lot is happening in a single line, which might make the code hard to read. Additionally, the fitted logistic regression model isn't stored in any variable, so we can't inspect it or use it to predict on any other data**

NAIVE BAYES CLASSIFIERS



Naive Bayes classifiers are a family of classifiers that are quite similar to the linear models



They tend to be even faster in training. The price paid for this efficiency is that naive Bayes models *often provide generalization performance* that is *slightly worse than that of linear classifiers like LogisticRegression and LinearSVC*.



The reason that naive Bayes models are so efficient is that they learn parameters by looking at each feature individually and collecting simple per-class statistics from each feature.

THERE ARE THREE KINDS OF NAIVE BAYES CLASSIFIERS IMPLEMENTED IN *SCIKIT-LEARN*:



GaussianNB

BernoulliNB

MultinomialNB

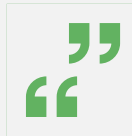
GAUSSIANNB, BERNOULLINB, AND MULTINOMIALNB



GaussianNB can be applied to any continuous data,



While BernoulliNB assumes binary data and



MultinomialNB assumes count data (that is, that each feature represents an integer count of something, like how often a word appears in a sentence).



BernoulliNB and MultinomialNB are mostly used in text data classification.

BERNOULLINB

- The *BernoulliNB* classifier counts how often every feature of each class is not zero. This is most easily understood with an example:

```
X = np.array([[0, 1, 0, 1],  
              [1, 0, 1, 1],  
              [0, 0, 0, 1],  
              [1, 0, 1, 0]])
```

```
y = np.array([0, 1, 0, 1])
```

- Here, we have four data points, with four binary features each. There are two classes, 0 and 1.
- For class 0 (the first and third data points), the first feature is zero two times and nonzero zero times, the second feature is zero one time and nonzero one time, and so on.
- These same counts are then calculated for the data points in the second class.

COUNTING THE NONZERO ENTRIES PER CLASS IN ESSENCE, LOOKS LIKE THIS:

- `counts = {}`
- `for label in np.unique(y):`
- `# iterate over each class`
- `# count (sum) entries of 1 per feature`
- `counts[label] = X[y == label].sum(axis=0)`
- `print("Feature counts:\n{}".format(counts))`
- `Feature counts:`
- `{0: array([0, 1, 0, 2]), 1: array([2, 0, 2, 1])}`

SUMMARY

The other two naive Bayes models, MultinomialNB and GaussianNB, are slightly different in what kinds of statistics they compute.



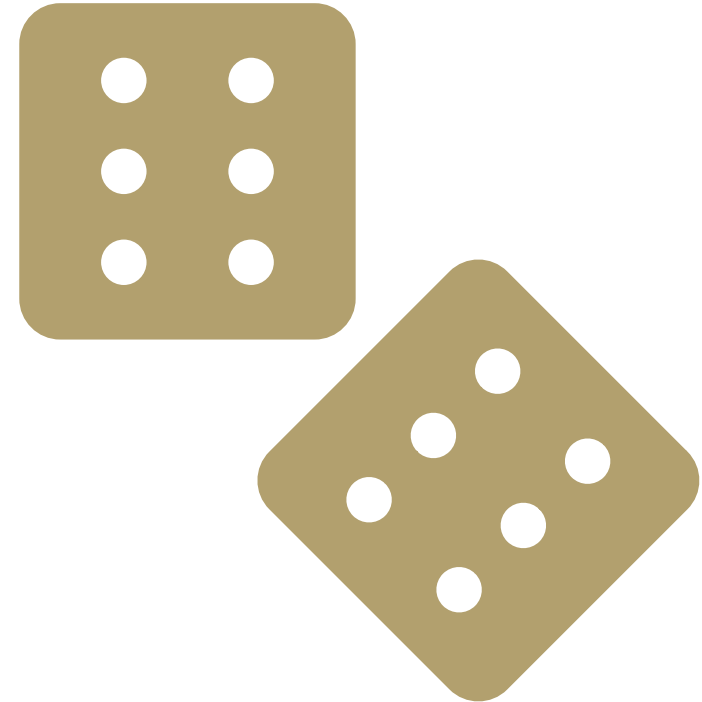
MultinomialNB takes into account the average value of each feature for each class, while



GaussianNB stores the average value as well as the standard deviation of each feature for each class.

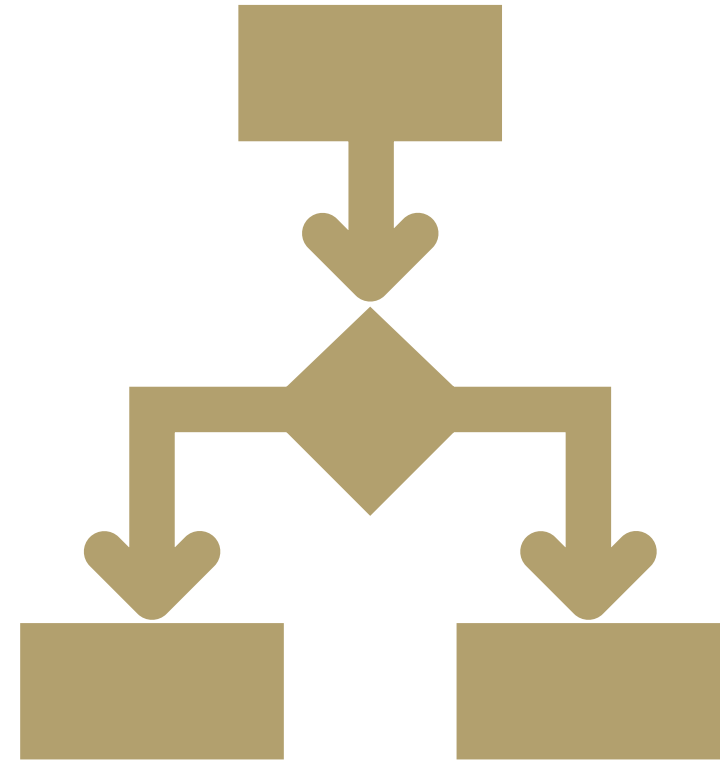
SUMMARY

- To make a prediction, a data point is compared to the statistics for each of the classes, and the best matching class is predicted. Interestingly, for both MultinomialNB and BernoulliNB, this leads to a prediction formula that is of the same form as in the linear models.
- Unfortunately, `coef_` for the naive Bayes models has a somewhat different meaning than in the linear models in that `coef_` is not the same as `w`.



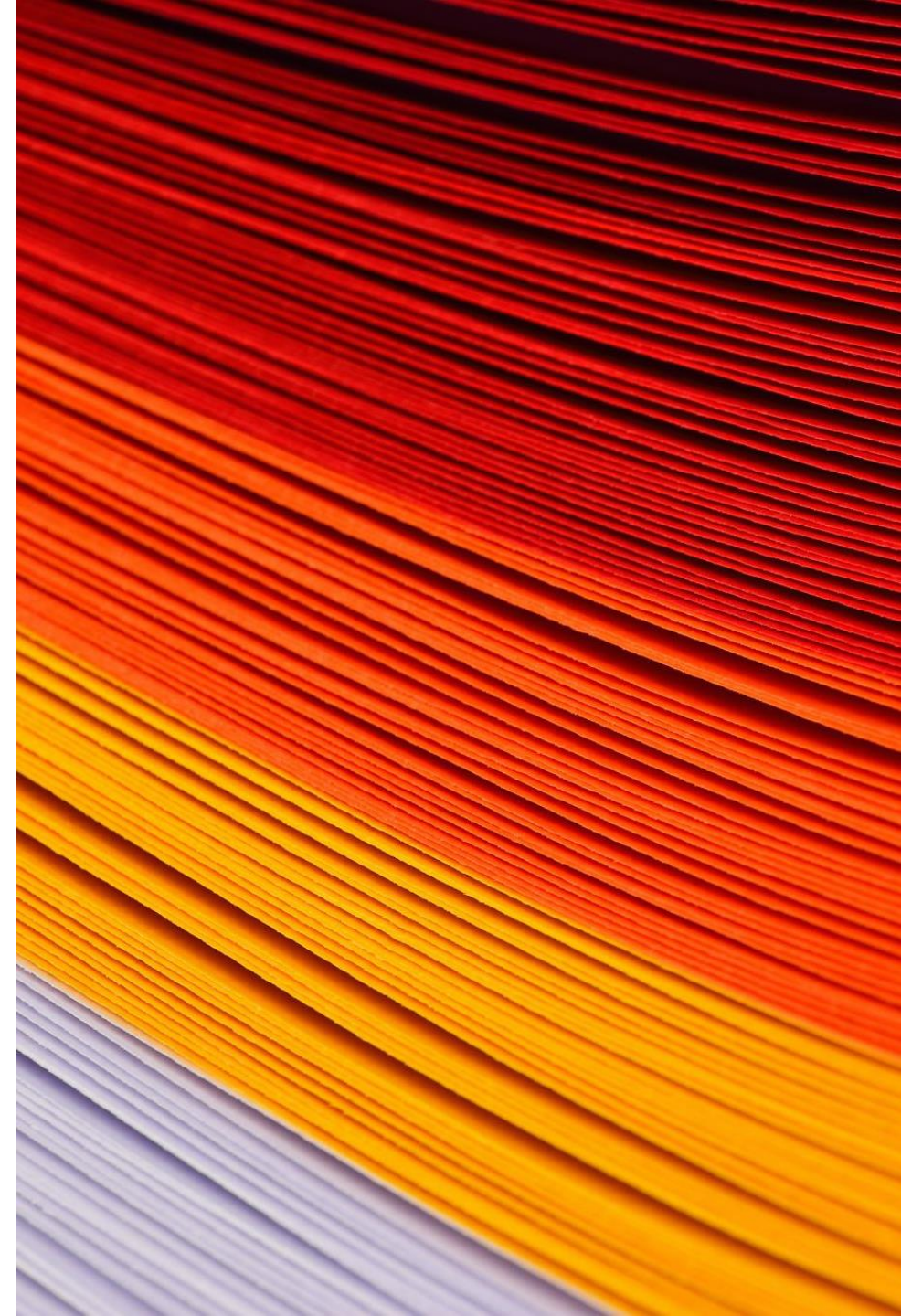
STRENGTHS, WEAKNESSES, AND PARAMETERS OF NB MODELS

- MultinomialNB and BernoulliNB have a single parameter, alpha, which controls model complexity.
- The way alpha works is that the algorithm adds to the data alpha many virtual data points that have positive values for all the features.
- This results in a “smoothing” of the statistics.
- A large alpha means more smoothing, resulting in less complex models.
- The algorithm’s performance is relatively robust to the setting of alpha, meaning that setting alpha is not critical for good performance.
- However, tuning it usually improves accuracy somewhat.



STRENGTHS

- GaussianNB is mostly used on very high-dimensional data, while the other two variants of naive Bayes are widely used for sparse count data such as text. MultinomialNB usually performs better than BinaryNB, particularly on datasets with a relatively large number of nonzero features (i.e., large documents).



WEAKNESS/WEAKNESS



The naive Bayes models share many of the strengths and weaknesses of the linear models.



They are very fast to train and to predict, and the training procedure is easy to understand.

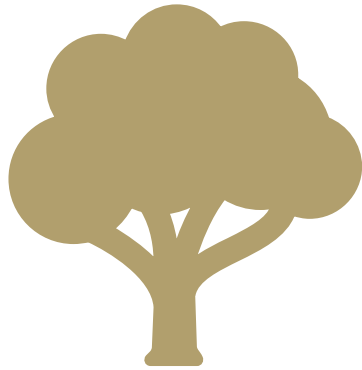


The models work very well with high-dimensional sparse data and are relatively robust to the parameters.



Naive Bayes models are great baseline models and are often used on very large datasets, where training even a linear model might take too long.

DECISION TREES



Decision trees are widely used models for classification and regression tasks.

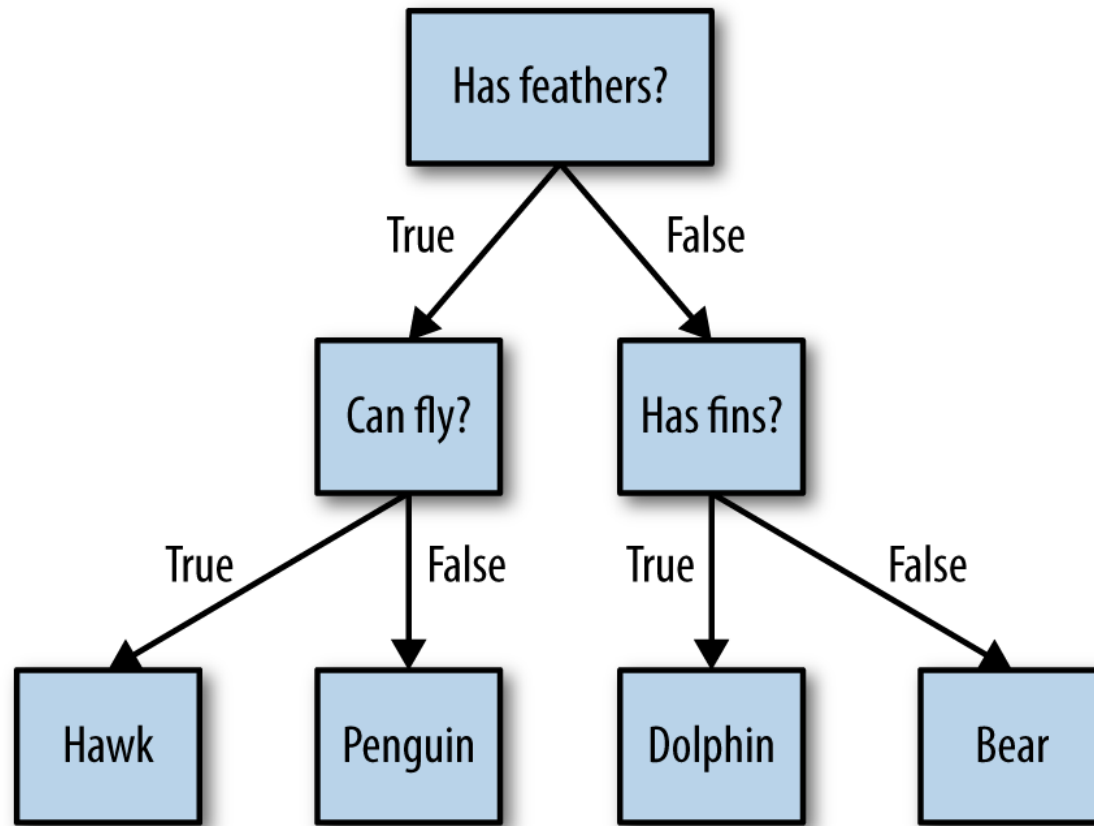


Essentially, they learn a hierarchy of if/else questions, leading to a decision.

DECISION TREES

- These questions are similar to the questions you might ask in a game of 20 Questions.
- Imagine you want to distinguish between the following four animals: bears, hawks, penguins, and dolphins.

A DECISION TREE TO DISTINGUISH AMONG SEVERAL ANIMALS



Each node in the tree either represents a question or a terminal.

Node (also called a leaf) that contains the answer.

The edges connect the answers to a question with the next question you would ask.

BUILDING DECISION TREES